

# FILE HANDLING IN JAVA

```
import java.io.File;
class A
{
    public static void main(String[] args)
    {
        // File name specified
        File obj = new File("myfile.txt");
        System.out.println("File Created!");
    }
}
```

## Streams in Java

In Java, a sequence of data is known as a stream. This concept is used to perform I/O operations on a file. Below are the **types of Streams**:

### 1. Input Stream

The Java InputStream class is the superclass of all input streams. The input stream is used to read data from numerous input devices like the keyboard, network, etc. InputStream is an abstract class, and because of this, it is not useful by itself. However, its subclasses are used to read data.

There are several subclasses of the InputStream class, which are as follows:

1. AudioInputStream
2. ByteArrayInputStream
3. FileInputStream
4. FilterInputStream
5. StringBufferInputStream
6. ObjectInputStream
7. **Common Methods of InputStream:**

| Method               | Description                                                             |
|----------------------|-------------------------------------------------------------------------|
| read()               | Reads one byte of data from the input stream.                           |
| read(byte[] array)() | Reads byte from the stream and stores that byte in the specified array. |

| Method          | Description                                                                       |
|-----------------|-----------------------------------------------------------------------------------|
| mark()          | It marks the position in the input stream until the data has been read.           |
| available()     | Returns the number of bytes available in the input stream.                        |
| markSupported() | It checks if the mark() method and the reset() method is supported in the stream. |
| reset()         | Returns the control to the point where the mark was set inside the stream.        |
| skips()         | Skips and removes a particular number of bytes from the input stream.             |
| close()         | Closes the input stream.                                                          |

## 2. Output Stream

The output stream is used to write data to numerous output devices like the monitor, file, etc. OutputStream is an abstract superclass that represents an output stream. OutputStream is an abstract class and because of this, it is not useful by itself. However, its subclasses are used to write data.

There are several subclasses of the OutputStream class which are as follows:

1. ByteArrayOutputStream
2. FileOutputStream
3. StringBufferOutputStream
4. ObjectOutputStream
5. DataOutputStream
6. PrintStream
7. **Common Methods of OutputStream:**

| Method  | Description                                     |
|---------|-------------------------------------------------|
| write() | Writes the specified byte to the output stream. |

| Method                           | Description                                                                  |
|----------------------------------|------------------------------------------------------------------------------|
| <code>write(byte[] array)</code> | Writes the bytes which are inside a specific array to the output stream.     |
| <code>close()</code>             | Closes the output stream.                                                    |
| <code>flush()</code>             | Forces to write all the data present in an output stream to the destination. |

## Java File Class Methods

The following table depicts several File Class methods:

| Method Name                         | Description                                     | Return Type |
|-------------------------------------|-------------------------------------------------|-------------|
| <b><code>canRead()</code></b>       | It tests whether the file is readable or not.   | Boolean     |
| <b><code>canWrite()</code></b>      | It tests whether the file is writable or not.   | Boolean     |
| <b><code>createNewFile()</code></b> | It creates an empty file.                       | Boolean     |
| <b><code>delete()</code></b>        | It deletes a file.                              | Boolean     |
| <b><code>exists()</code></b>        | It tests whether the file exists or not.        | Boolean     |
| <b><code>length()</code></b>        | Returns the size of the file in bytes.          | Long        |
| <b><code>getName()</code></b>       | Returns the name of the file.                   | String      |
| <b><code>list()</code></b>          | Returns an array of the files in the directory. | String[]    |
| <b><code>mkdir()</code></b>         | Creates a new directory.                        | Boolean     |

| Method Name              | Description                                | Return Type |
|--------------------------|--------------------------------------------|-------------|
| <b>getAbsolutePath()</b> | Returns the absolute pathname of the file. | String      |

## File Operations

The following are the several operations that can be performed on a file in Java:

- Create a File
- Read from a File
- Write to a File
- Delete a File

### Create a File

- In order to create a file in Java, you can use the `createNewFile()` method.
- If the file is successfully created, it will return a Boolean value `true` and `false` if the file already exists.

**Example:**

*// Creating File using Java Program*

*// Import the File class*

**import java.io.File;**

**import java.io.IOException;**

**public class CreateFile**

{

**public static** void main(String[] args)

    {

*// Creating the File also*

*// Handling Exception*

**try** {

            File Obj = **new** File("myfile.txt");

*// Creating File*

**if** (Obj.createNewFile()) {

            System.out.println("File created: " + Obj.getName());

        }

```

        else {
            System.out.println("File already exists.");
        }
    }

    // Exception Thrown
    catch (IOException e) {
        System.out.println("An error has occurred.");
        e.printStackTrace();
    }
}
}

```

## Write to a File

We use the `FileWriter` class along with its `write()` method in order to write some text to the file.

### Example:

*// Writing Files using Java Program*

*// Import the FileWriter class*

```
import java.io.FileWriter;
```

```
import java.io.IOException;
```

```
public class WriteFile
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        // Writing Text File also
```

```
        // Exception Handling
```

```
        try {
```

```
            FileWriter Writer = new FileWriter("myfile.txt");
```

```
            // Writing File
```

```
            Writer.write("Files in Java are seriously good!!");
```

```
            Writer.close();
```

```

        System.out.println("Successfully written.");
    }

    // Exception Thrown
    catch (IOException e) {
        System.out.println("An error has occurred.");
        e.printStackTrace();
    }
}
}

```

## Delete a File

We use the delete() method in order to delete a file.

### Example:

*// Deleting File using Java Program*

```
import java.io.File;
```

```
public class DeleteFile
```

```

{
    public static void main(String[] args)
    {
        File Obj = new File("myfile.txt");

        // Deleting File
        if (Obj.delete()) {
            System.out.println("The deleted file is : " + Obj.getName());
        }
        else {
            System.out.println(
                "Failed in deleting the file.");
        }
    }
}

```

```
STRING
```

```

class Main {
    public static void main(String[] args) {
        String str1 = "Learn Java";
    }
}

```

```
String str2 = "Learn Kolin";
int result;

// comparing str1 with str2
result = str1.compareTo(str2);

System.out.println(result);
}
}

// Output: -1
```

### **Example: Java String compareToIgnoreCase()**

```
class Main {
    public static void main(String[] args) {
        String str1 = "Learn Java";
        String str2 = "learn java";
        String str3 = "Learn Kolin";
        int result;

        // comparing str1 with str2
        result = str1.compareToIgnoreCase(str2);
        System.out.println(result); // 0

        // comparing str1 with str3
        result = str1.compareToIgnoreCase(str3);
        System.out.println(result); // -1

        // comparing str3 with str1
        result = str3.compareToIgnoreCase(str1);
        System.out.println(result); // 1
    }
}
```

### **Example 2: Check if Two Strings are Equal**

```
class Main {
    public static void main(String[] args) {
```

```

String str1 = "LEARN JAVA";
String str2 = "Learn Java";

// if str1 and str2 are equal (ignoring case differences),
// the result is 0
if (str1.compareToIgnoreCase(str2) == 0) {
    System.out.println("str1 and str2 are equal");
}
else {
    System.out.println("str1 and str2 are not equal");
}
}
}

```

### **Java String length()**

```

class Main {
    public static void main(String[] args) {
        String str1 = "Java is fun";

        // returns the length of str1
        int length = str1.length();

        System.out.println(str1.length());
    }
}

```

### **Java String replace()**

```

class Main {
    public static void main(String[] args) {
        String str1 = "bat ball";

        // replace b with c
        System.out.println(str1.replace('b', 'c'));

    }
}

```

// Output: cat call

## **Java String indexOf()**

```
class Main {  
    public static void main(String[] args) {  
        String str1 = "Java is fun";  
        int result;  
  
        // getting index of character 's'  
        result = str1.indexOf('s');  
  
        System.out.println(result);  
    }  
}
```

// Output: 6

## **Java String trim()**

```
lass Main {  
    public static void main(String[] args) {  
        String str1 = "  Learn Java Programming  ";  
  
        System.out.println(str1.trim());  
  
    }  
}
```

// Output: Learn Java Programming

## **Java String toLowerCase()**

```
class Main {  
    public static void main(String[] args) {  
        String str1 = "JAVA PROGRAMMING";  
  
        // convert to lower case letters  
        System.out.println(str1.toLowerCase());  
    }  
}
```

```
}  
}
```

// Output: java programming

## Java String concat()

```
class Main {  
    public static void main(String[] args) {  
        String str1 = "Java";  
        String str2 = "Programming";  
  
        // concatenate str1 and str2  
        System.out.println(str1.concat(str2));  
  
    }  
}
```

// Output: JavaProgramming

## Java String isEmpty()

```
class Main {  
    public static void main(String[] args) {  
  
        String str1 = "Java Programming";  
        String str2 = "";  
  
        System.out.println(str1.isEmpty()); // false  
        System.out.println(str2.isEmpty()); // true  
  
    }  
}
```

# ARRAYLIST

Java ArrayList is a part of the [collections framework](#) and it is a class of java.util package. It provides us with dynamic-sized arrays in Java.

- The main advantage of ArrayList is that, unlike normal arrays, we don't need to mention the size when creating ArrayList. It automatically adjusts its capacity as elements are added or removed.
- It may be slower than standard arrays, but it is helpful when the size is not known in advance. Note that creating a large fixed-sized array would cause a waste of space.
- Since ArrayList is part of the collections framework, it has better interoperability with other collections. For example, conversion to a HashSet is straightforward.
- With generics, ArrayList<T> ensures type safety at compile-time.
- **Syntax of ArrayList**
- *ArrayList<Integer> arr = new ArrayList<Integer>();*

- 

EG.

```
import java.util.ArrayList;
```

```
class Main {  
    public static void main (String[] args) {  
  
        // Creating an ArrayList  
        ArrayList<Integer> a = new ArrayList<Integer>();  
  
        // Adding Element in ArrayList  
        a.add(1);  
        a.add(2);  
        a.add(3);  
  
        // Printing ArrayList  
        System.out.println(a);  
  
    }  
}
```

```
}
```

## Constructors in ArrayList in Java

In order to Create an ArrayList, we need to create an object of the ArrayList class. The ArrayList class consists of various [constructors](#) which allow the possible creation of the array list. The following are the constructors available in this class:

| Constructor                    | Description                                                                                          | Initialize and Declare ArrayList         |
|--------------------------------|------------------------------------------------------------------------------------------------------|------------------------------------------|
| <b>ArrayList()</b>             | This constructor is used to build an empty array list.                                               | <b>ArrayList arr = new ArrayList();</b>  |
| <b>ArrayList(Collection c)</b> | This constructor is used to build an array list initialized with the elements from the collection c. | <b>ArrayList arr = new ArrayList(c);</b> |
| <b>ArrayList(int capacity)</b> | This constructor is used to build an array list with the initial capacity being specified.           | <b>ArrayList arr = new ArrayList(N);</b> |

### Example

```
// Java Program Example to Demonstrate  
// Addition, Deletion and Updation of Element  
import java.util.*;
```

```
class Main {  
    public static void main(String args[]){  
  
        // Creating an Array of string type  
        ArrayList<String> al = new ArrayList<>();  
  
        // 1. Addition  
  
        // Adding elements to ArrayList  
        // at the end
```

```
al.add("Hello");
al.add("Hello");

    System.out.println("Original List : "+al);

    // Adding Elements at the specific
    // index
al.add(1, "For");

    System.out.println("After Adding element at index 1 : "+ al);

    // 2. Deletion of Element

    // Removing Element using index
al.remove(0);

    System.out.println("Element removed from index 0 : "+ al);

    // Removing Element using the value
al.remove("Hello");

    System.out.println("Element Geeks removed : "+ al);

    // 3. Updating Values

    // Updating value at index 0
al.set(0, "GFG");

    // Printing all the elements in an ArrayList
    System.out.println("List after updation of value : "+al);
}
}
```